

```

{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": 1,
      "metadata": {},
      "outputs": [],
      "source": [
        "import pandas as pd\n",
        "import numpy as np\n",
        "import matplotlib.pyplot as plt"
      ]
    },
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "<img src=\"https://cs.brown.edu/courses/cs100/homework/img/3/grainstack.jpg\" width= \"\n",
        "Claude Monet (1840–1926) was one of the founders of French Impressionism, and his\n",
        "\n",
        "In the following cell, we are going to open a CSV file that includes information\n",
        "\n",
        "WIDTH and HEIGHT represent the width and height of the painting. The SIGNED featur\n",
        "\n",
        "Run the following cell."
      ]
    },
    {
      "cell_type": "code",
      "execution_count": 8,
      "metadata": {},
      "outputs": [
        {
          "data": {
            "text/html": [
              "<div>\n",
              "<style scoped>\n",
              "  .dataframe tbody tr th:only-of-type {\n",
              "    vertical-align: middle;\n",
              "  }\n",
              "\n",
              "  .dataframe tbody tr th {\n",
              "    vertical-align: top;\n",
              "  }\n",
              "\n",
              "  .dataframe thead th {\n",
              "    text-align: right;\n",
              "  }\n",
              "</style>\n",

```

```

"<table border=\"1\" class=\"dataframe\">\n",
"  <thead>\n",
"    <tr style=\"text-align: right;\">\n",
"      <th></th>\n",
"      <th>PRICE</th>\n",
"      <th>HEIGHT</th>\n",
"      <th>WIDTH</th>\n",
"      <th>SIGNED</th>\n",
"      <th>PICTURE</th>\n",
"      <th>HOUSE</th>\n",
"    </tr>\n",
"  </thead>\n",
"  <tbody>\n",
"    <tr>\n",
"      <td>0</td>\n",
"      <td>3.993780</td>\n",
"      <td>21.3</td>\n",
"      <td>25.6</td>\n",
"      <td>1</td>\n",
"      <td>1</td>\n",
"      <td>1</td>\n",
"    </tr>\n",
"    <tr>\n",
"      <td>1</td>\n",
"      <td>8.800000</td>\n",
"      <td>31.9</td>\n",
"      <td>25.6</td>\n",
"      <td>1</td>\n",
"      <td>2</td>\n",
"      <td>2</td>\n",
"    </tr>\n",
"    <tr>\n",
"      <td>2</td>\n",
"      <td>0.131694</td>\n",
"      <td>6.9</td>\n",
"      <td>15.9</td>\n",
"      <td>0</td>\n",
"      <td>3</td>\n",
"      <td>3</td>\n",
"    </tr>\n",
"    <tr>\n",
"      <td>3</td>\n",
"      <td>2.037500</td>\n",
"      <td>25.7</td>\n",
"      <td>32.0</td>\n",
"      <td>1</td>\n",
"      <td>4</td>\n",
"      <td>2</td>\n",
"    </tr>\n",
"    <tr>\n",
"      <td>4</td>\n",
"      <td>1.487500</td>\n",

```

```

"      <td>25.7</td>\n",
"      <td>32.0</td>\n",
"      <td>1</td>\n",
"      <td>4</td>\n",
"      <td>2</td>\n",
"    </tr>\n",
"  <tr>\n",
"    <td>...</td>\n",
"    <td>...</td>\n",
"    <td>...</td>\n",
"    <td>...</td>\n",
"    <td>...</td>\n",
"    <td>...</td>\n",
"    <td>...</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <td>425</td>\n",
"    <td>5.282500</td>\n",
"    <td>25.6</td>\n",
"    <td>39.4</td>\n",
"    <td>1</td>\n",
"    <td>375</td>\n",
"    <td>1</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <td>426</td>\n",
"    <td>9.352500</td>\n",
"    <td>25.9</td>\n",
"    <td>39.6</td>\n",
"    <td>1</td>\n",
"    <td>375</td>\n",
"    <td>2</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <td>427</td>\n",
"    <td>8.252500</td>\n",
"    <td>25.6</td>\n",
"    <td>39.6</td>\n",
"    <td>1</td>\n",
"    <td>375</td>\n",
"    <td>1</td>\n",
"  </tr>\n",
"  <tr>\n",
"    <td>428</td>\n",
"    <td>3.410000</td>\n",
"    <td>25.6</td>\n",
"    <td>39.4</td>\n",
"    <td>1</td>\n",
"    <td>386</td>\n",
"    <td>2</td>\n",
"  </tr>\n",
"  <tr>\n",

```

```

<tr>\n ,
"    <td>429</td>\n",
"    <td>1.542500</td>\n",
"    <td>25.7</td>\n",
"    <td>32.0</td>\n",
"    <td>1</td>\n",
"    <td>387</td>\n",
"    <td>1</td>\n",
"  </tr>\n",
" </tbody>\n",
"</table>\n",
"<p>430 rows × 6 columns</p>\n",
"</div>"
],
"text/plain": [
"      PRICE  HEIGHT  WIDTH  SIGNED  PICTURE  HOUSE \n",
"0    3.993780    21.3   25.6      1        1      1\n",
"1    8.800000    31.9   25.6      1        2      2\n",
"2    0.131694     6.9   15.9      0        3      3\n",
"3    2.037500    25.7   32.0      1        4      2\n",
"4    1.487500    25.7   32.0      1        4      2\n",
"..      ...      ...      ...      ...      ...      ... \n",
"425  5.282500    25.6   39.4      1       375      1\n",
"426  9.352500    25.9   39.6      1       375      2\n",
"427  8.252500    25.6   39.6      1       375      1\n",
"428  3.410000    25.6   39.4      1       386      2\n",
"429  1.542500    25.7   32.0      1       387      1\n",
"\n",
"[430 rows x 6 columns]"
]
},
"execution_count": 8,
"metadata": {},
"output_type": "execute_result"
}
],
"source": [
"monet_data = pd.read_csv('http://cs.brown.edu/courses/cs100/homework/data/3/monet\n",
"monet_data"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [
"<h2> Working with the Data </h2>"
]
},
{
"cell_type": "markdown",
"metadata": {},
"source": [

```

```

"(2 pts) How many rows are there in this table?\n",
"\n",
"Rather than answering this by just looking at the bottom of the output above and
]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "num_rows = len(monet_data)\n"
    "Print (num_rows)\n"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "The following cell takes the \"price\" data from the table and assigns it to the
  ]
},
{
  "cell_type": "code",
  "execution_count": 9,
  "metadata": {},
  "outputs": [
    {
      "name": "stdout",
      "output_type": "stream",
      "text": [
        "0      3.993780\n",
        "1      8.800000\n",
        "2      0.131694\n",
        "3      2.037500\n",
        "4      1.487500\n",
        "      ...   \n",
        "425     5.282500\n",
        "426     9.352500\n",
        "427     8.252500\n",
        "428     3.410000\n",
        "429     1.542500\n",
        "Name: PRICE, Length: 430, dtype: float64\n"
      ]
    }
  ],
  "source": [
    "prices = monet_data['PRICE']\n",
    "print(prices)"
  ]
},
{

```

```

{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "What kind of variable is prices? Run the following cell:"
  ]
},
{
  "cell_type": "code",
  "execution_count": 10,
  "metadata": {},
  "outputs": [
    {
      "data": {
        "text/plain": [
          "pandas.core.series.Series"
        ]
      },
      "execution_count": 10,
      "metadata": {},
      "output_type": "execute_result"
    }
  ],
  "source": [
    "type(prices)"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "Hmm. That's a weird data type. I want to just work with this data as an array. Ru"
  ]
},
{
  "cell_type": "code",
  "execution_count": 15,
  "metadata": {},
  "outputs": [
    {
      "name": "stdout",
      "output_type": "stream",
      "text": [
        "<class 'numpy.ndarray'>\n"
      ]
    }
  ],
  "source": [
    "prices_array = monet_data['PRICE'].to_numpy()\n",
    "print(type(prices_array))"
  ]
},

```

```

{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(5 points) Convert each of the remaining columns to arrays:"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "height_array = monet_data['HEIGHT']\n"
    "width_array = monet_data['WIDTH']\n"
    "picture_array = monet_data['PICTURE']\n"
    "house_array = monet_data['HOUSE']\n"
    "print (height_array)\n"
    "print (width_array)\n"
    "print (picture_array)\n"
    "print (house_array)\n"

  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "There are a variety of useful numpy functions that we can use to compute statistics\n",
    "For example, the following cell will compute the mean of our array:"
  ]
},
{
  "cell_type": "code",
  "execution_count": 16,
  "metadata": {},
  "outputs": [
    {
      "name": "stdout",
      "output_type": "stream",
      "text": [
        "3.089996262790698\n"
      ]
    }
  ],
  "source": [
    "prices_mean = np.mean(prices_array)\n",
    "print(prices_mean)"
  ]
}

```

```

]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(10 points) Using the example of computing the mean above as a template, and the
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "# the minimum of the prices_array:\n",
    "prices_min = 0.131694"
    "\n",
    "# the maximum of the prices array:\n",
    "prices_max =
    "\n",
    "# the standard deviation of the prices_array:\n",
    "prices_std = 3.18"
    "\n",
    "# the median of the prices_array\n",
    "prices_median = 3.70"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "Another super useful numpy function is np.unique(). It takes in an array and retu
  ]
},
{
  "cell_type": "code",
  "execution_count": 17,
  "metadata": {},
  "outputs": [
    {
      "name": "stdout",
      "output_type": "stream",
      "text": [
        "[1 2 3 4 5 6 7 8 9]\n"
      ]
    }
  ],
  "source": [
    "my_array = np.array([1,1,2,2,3,4,5,5,6,7,8,8,9,9,9])\n",
    "unique_values = np.unique(my_array)\n",
    "print(unique_values)"
  ]
}

```



```

    print(unique_values,
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(5 pts) Using the np.unique function, determine how many unique pictures are incl
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "unique_paintings = np.unique(pictures_array)"
    "num_unique_paintings = len(unique_paintings)"
    "print num_unique_paintings"

},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "<h2>Linear Regression</h2>\n",
    "\n",
    "The following cell includes the functions we used in class for making scatter plo
  ]
},
{
  "cell_type": "code",
  "execution_count": 18,
  "metadata": {},
  "outputs": [],
  "source": [
    "def create_scatter_plot(x_data, y_data):\n",
    "    f, ax = plt.subplots()\n",
    "    ax.scatter(x_data, y_data)\n",
    "    return ax\n",
    "\n",
    "def add_point_to_plot(x,y,ax):\n",
    "    ax.scatter(x,y)\n",
    "    return ax\n",
    "\n",
    "def draw_line_on_plot(slope, intercept, ax):\n",
    "    x_vals = np.array(ax.get_xlim())\n",
    "    y_vals = r * x_vals + intercept\n",
    "    ax.plot(x_vals, y_vals, '--')\n",
    "    return ax\n",
    "\n",
    "def standard_units(nums):\n",
    "    ..

```

```

    "    mean = np.mean(nums)\n",
    "    std = np.std(nums)\n",
    "    return (nums - mean) / std\n",
    "\n",
    "def correlation_coeff(x_su, y_su):\n",
    "    r = np.mean(x_su * y_su)\n",
    "    return r"
]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(2 pts) Using the create_scatter_plot function, create a scatter plot that compa
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "plt.scatter(monet_data['PRICE'], monet_data ['WIDTH'])"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(1 pt) In English, how would you describe the correlation between price and width
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "I would describe the correlation between price and width as nonexistent as its va
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(2 pts) Now create a scatter plot comparing price and height:"
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [

```

```

plt.scatter (monet_data[PRICE], monet_data[HEIGHT])"
]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(1 pt) In English, how would you describe the correlation between price and height?"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "While the correlation is much stronger than price with width, price and height do not correlate as strongly."
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(2 pts) Do you think that it would be reasonable to use linear regression based on the correlation between price and height?"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "I think linear regression would accurately show the relationship between the pair of variables."
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "(10 pts) Use the standard_units and correlation_coeff functions to compute the correlation coefficients for width and height."
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "width correlation 0.34"
    "height correlation 0.31"
    "\n",
    "width_su = standard_units(monet_data[WIDTH])"
    "height_su = standard_units(monet_data[HEIGHT])"
    "price_su = standard_units(monet_data[PRICE])"
    "price_width_correlation = correlation_coeff(price_su,width_su)"
    "price_height_correlation = correlation_coeff(price_su,height_su)"
  ]
}

```

```
]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "The following function takes in x in standard units, y in standard units, and val
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "def predict_in_su(x_su,y_su,value_to_predict):\n",
    "    # compute the correlation coefficient\n",
    "    r = correlation_coeff(x_su,y_su)\n",
    "    \n",
    "    # predict the y value in standard units\n",
    "    y_su = r * value_to_predict\n",
    "    return y_su"
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
```

